BreakCheck AI — Manual Testing Guide

Version: 1.0
Audience: SME reviewers and first-time testers
Purpose: Check the application from setup through the dashboard and REST service, using the project brief and policy documents as the answer key.

Contents

- What this guide does
- The system in one minute
- Before you start
- Expected code rules
- Start the application
- Backend query setup
- Manual tests A–F
- Decision request
- Expected errors
- Result sheet and sources

What this guide does

This is a black-box acceptance guide: follow the steps, use the running service and dashboard, and compare what you see with the expected results here. It does not ask you to read application code. It covers the visible outputs of the data pipeline, trained models, knowledge search, campaign checks, human decision gate, permissions, chat and audit trail.

Some outputs are intentionally different each run: assessment IDs, model scores, exposure values, timestamps and the assistant's wording. Compare the facts, status, counts, source documents and permissions listed here. Do not expect identical AI prose.

A campaign decision changes its saved record and cannot be undone in the dashboard. Before testing, write down the campaign references you plan to use. Use a different, not-yet-assessed campaign for each assessment and decision branch. The worked example in this guide can be assessed only once unless the SME provides a clean test database.

The system in one minute

An Executive opens a campaign and asks for a review. BreakCheck checks its copy notes and every booked spot against the campaign category's own code, then makes an assessment recommendation. It pauses with status `PENDING_DECISION`. A different Clearance Controller chooses one of the decisions that the spots allow. Only after that human decision are spots cleared or withdrawn and the agency documents created. Agency Liaisons can ask policy questions and see status for campaigns belonging to their assigned agencies.

Before you start

1. Open a terminal in the `Project` folder.

2. Install the listed packages once: `python -m pip install -r requirements.txt`.
3. Make sure the SME has provided valid demo passwords and an approved Gemini key is configured locally. Never put a password, API key or session token in this guide or in screenshots.
4. Keep one terminal open for the API and another for the dashboard. Leave both running during tests, except for the restart test.
5. Use the OpenAPI page at `http://localhost:8081/docs` if an API field or allowed enum is not shown in an example below. It is the service's interactive request guide.

Demo roles named in the brief

The brief's seed-account examples include `priya` and `owen` (Clearance Controllers), `tom` (Clearance Executive), `lucy` (Agency Liaison assigned QSC, HWK, PVT and FWL) and `nadia` (Agency Liaison assigned LNT, GRF, ASH and NLT). The PDF does not publish passwords; ask the SME for the approved test passwords. Do not guess them.

Expected code rules — use this as the chat and spot-check answer key

All times below are on the ordinary clock in the policy code. "In or next to" means a spot in a centre break or a break exactly between two programmes. The audience threshold is inclusive: an index **equal to or greater than** the figure breaks the rule.

Product category	Hours and live-sport exception	Audience and children's rule	Daily limit and spacing
Alcohol	No restricted hours; any time. No exception needed.	Index 120. In or next to a marked children's programme.	6 per campaign/channel/broadcast day; at least 20 minutes apart.
Betting and gaming	05:30–21:00. Only a centre break more than 10 minutes after live coverage starts and more than 10 minutes before it ends is exempt. An end break never qualifies.	Index 110. In or next to a marked children's programme.	7 per campaign/channel/broadcast day; at least 30 minutes apart.
Less healthy food	05:30–21:00. No live-sport exception.	Index 120. In or next to a marked children's programme.	8 per campaign/channel/broadcast day; at least 10 minutes apart.
Consumer credit	06:00–19:00, including live sport. No exception.	Index 125. In or next to a marked children's programme.	5 per campaign/channel/broadcast day; at least 15 minutes apart.
Medicines and health	No restricted hours; any time. No exception needed.	Index 150. In a marked children's programme only. An adjacent end break is not caught by this rule, but still has the audience-index rule.	10 per campaign/channel/broadcast day; at least 5 minutes apart.

The six checks always use **the booked category's own row**. A wrong threshold, a wrong children's rule or the betting exception applied to a different category is a failure. Source names are `Code-Alcohol.pdf`, `Code-Betting-And-Gaming.pdf`, `Code-Less-Healthy-Food.pdf`, `Code-Consumer-Credit.pdf` and `Code-Medicines-And-Health.pdf`.

Start the application in the required order

Run each command from `Project`. The setup and pipeline commands finish; the service and dashboard stay running.

```
python run_database_setup.py
python run_data_pipeline.py
python run_model_training.py
python run_knowledge_indexing.py
```

In terminal 1, start the service:

```
python run_api_service.py
```

Wait until it says the API is running on port `8081` and gives `http://localhost:8081/docs`. In terminal 2, also from `Project`, start the dashboard:

```
python -m streamlit run run_dashboard.py
```

Open `http://localhost:8080`. The sign-in page should appear. If the API cannot start, first check the error for the missing generated file and run the earlier setup step that produces it.

To inspect the generated evidence from the same `Project` terminal:

```
$Quality = Get-Content .\output\data_quality_report.json -Raw | ConvertFrom-Json
$Quality.PSObject.Properties.Name
$Metrics = Get-Content .\output\model_metrics.json -Raw | ConvertFrom-Json
$Metrics.amendment | Format-List *
$Metrics.exposure | Format-List *
$Metrics.issue | Format-List *
Get-ChildItem .\output\models
```

Expected: quality report lists the five cleaned extract names; the metrics show amendment, exposure and issue with scores, row counts and `meets_minimum`; model files are present. Scores themselves can vary.

Backend query setup - sign in once, then copy and paste

These commands are for Windows PowerShell. They ask for the demo username and password, sign in, and keep the session token in a variable. **You do not need to copy the token by hand.** Keep this PowerShell window open while using the commands. The header name is read from the running API's OpenAPI description, so the script does not guess it.

First, read the session-header name and define a sign-in helper:

```
$Api = "http://localhost:8081"
$Spec = Invoke-RestMethod -Uri "$Api/openapi.json"
$QueueOperation = $Spec.paths.PSObject.Properties["/queue"].Value.get
$HeaderParameter = $QueueOperation.parameters |
  Where-Object { $_.in -eq "header" } | Select-Object -First 1
if (-not $HeaderParameter) { throw "Open /docs and find the session-token header for GET /queue." }
$TokenHeader = $HeaderParameter.name
function Connect-BreakCheck {
  $Username = Read-Host "Demo username"
  $Password = Read-Host "SME-approved demo password"
  $LoginBody = @{ username = $Username; password = $Password } | ConvertTo-Json
  $Login = Invoke-RestMethod -Method Post -Uri "$Api/auth/signin" `
    -ContentType "application/json" -Body $LoginBody
  $HeadersForUser = @{}
  $HeadersForUser[$TokenHeader] = $Login.session_token
  [PSCustomObject]@{ Login = $Login; Headers = $HeadersForUser }
}
```

Sign in as the Executive first:

```
$ExecutiveSession = Connect-BreakCheck
$ExecutiveLogin = $ExecutiveSession.Login
$ExecutiveHeaders = $ExecutiveSession.Headers
$Headers = $ExecutiveHeaders
$ExecutiveLogin | Select-Object username, display_name, role, assigned_agencies, pages, areas
```

Expected: a successful reply identifies the signed-in name and correct role. A Liaison also has their four assigned agency codes; desk roles have no agency assignment. A wrong username and a wrong password both give the same `401` sign-in error.

When a different role is needed, keep the first role's headers and create another session like this:

```
$ControllerSession = Connect-BreakCheck
$ControllerHeaders = $ControllerSession.Headers
$Headers = $ControllerHeaders
```

Use `LiaisonSession` / `LiaisonHeaders` the same way for a Liaison. Do not sign in twice as the same user while expecting the first session to remain valid: a new sign-in replaces that user's old token. To sign out the current `$Headers` session, run:

```
Invoke-RestMethod -Method Post -Uri "$Api/auth/signout" -Headers $Headers
```

Useful request shapes

Paste a request into the same PowerShell window after sign-in. The query uses the exact category spelling from the brief:

```
$Queue = Invoke-RestMethod -Method Get `
  -Uri "$Api/queue?product_category=Betting%20and%20gaming" -Headers $Headers
$Queue | Select-Object -First 5 queue_position, campaign_reference, product_category, spots_booked,
spots_breaching
```

The API category filter accepts only the five categories or the “all” option shown by `/docs`. For the all-campaign count, use that all option from the `/docs` dropdown. There should be 240 live campaigns total. Category filtering must retain each campaign’s original full-queue position.

```
$Detail = Invoke-RestMethod -Method Get `
  -Uri "$Api/queue/CLR-QSC-0042117" -Headers $Headers
$Detail.summary | Format-List
$Detail.spot_results | Format-Table spot_reference, broadcast_day, break_time, breach_codes
```

To ask the assistant:

```
$Body = @{ question = "What is the under-18 index threshold for betting and gaming, and what is its live-sport
exception?" } | ConvertTo-Json
$Answer = Invoke-RestMethod -Method Post -Uri "$Api/chat/ask" `
  -Headers $Headers -ContentType "application/json" -Body $Body
$Answer | Format-List question_type, campaign_reference, answer, source_documents, was_refused
```

To see your latest ten questions:

```
Invoke-RestMethod -Method Get -Uri "$Api/chat/turns?turn_limit=10" -Headers $Headers |
  Format-Table question_type, campaign_reference, was_refused, question_text, answer_text
```

To see assessments waiting for a Controller and the recent organization-wide chat audit (use a role with audit access):

```
Invoke-RestMethod -Method Get -Uri "$Api/clearances?status_value=PENDING_DECISION" -Headers $Headers |
  Format-List *
Invoke-RestMethod -Method Get -Uri "$Api/audit/chat-turns?turn_limit=10" -Headers $Headers |
  Format-List *
```

`/docs` also has **Try it out** buttons for each request. Choose the route, fill the listed fields, and press **Execute**. Use the login response's `session_token` through the authorization/header control that `/docs` shows. PowerShell above keeps that step automatic.

`Invoke-RestMethod` prints the returned JSON body. For the exact HTTP status, read the code shown by `/docs` after **Execute**; a successful assessment must say `201`, and successful ordinary requests say `200`.

The 84 manual test cases

Record **Pass** or **Fail** beside every number. If a check fails, save the request/response (remove passwords and tokens first), campaign reference and time for the SME.

A. Setup, data cleaning, models and search index

1. **First database setup.** Run `python run_database_setup.py`. Pass: database is created with six tables—`agent_runs`, `chat_turns`, `clearance_assessments`, `controller_decisions`, `llm_calls`, `user_accounts`—and 14 demo accounts.
2. **Safe second setup.** Run the same command again. Pass: no failure and no duplicate accounts; expected summary is 0 created and 14 already present.
3. **All five cleaning jobs.** Run `python run_data_pipeline.py`. Expected received → kept counts: submissions train 30,706 → 29,697; submissions eval 8,514 → 8,224; audience train 30,877 → 29,817; audience eval 7,004 → 6,770; notes 34,727 → 33,045.
4. **Rows removed are explained.** Pass: removed counts are respectively 1,009, 290, 1,060, 234 and 1,682; the run writes `output/data_quality_report.json` and names the report in its final line.
5. **Before/after evidence.** Open the quality report. Pass: each of the five extract names has both an arrival profile and a cleaned profile; duplicate, missing-answer, impossible-answer and too-short-note removals are accounted for where they apply.

6. **Spelling, case and whitespace.** Check the cleaned tables against the raw samples in the brief. Pass: aliases and case variants become the category/channel/day/source/flag spellings defined by the brief; cleaned text has no extra leading or trailing spaces; unknown notations are reported as errors, not silently invented.
7. **True missing values stay missing.** Pass: complete null tokens such as `N/A`, `NULL`, `Not recorded` and blank cells become missing values; an ordinary word that merely contains one of those tokens is kept. Empty offer details, non-credit APR and film episode number remain empty.
8. **Length and numeric conversion.** Pass: `30"`, `30s`, `30 sec`, `0:30` and `00:00:30` mean 30 seconds; an audience such as `0.2272m` becomes 227.2 thousand, `62.3k` becomes 62.3 thousand and `220,500 viewers` becomes 220.5 thousand. The copy clock number is authoritative if its length disagrees with the written length.
9. **Notes are cleaned.** Pass: markup is removed, escaped characters display normally, no-break spaces become ordinary spaces, repeated whitespace is reduced and note text is readable. The sample note `Noted, thanks.` is too short to train on and is set aside.
10. **Broadcast clock conversion.** Pass: Sunday 05:40 on an ordinary calendar clock becomes Saturday 29:40; Monday 01:10 becomes Sunday 25:10; Saturday 23:30–00:30 becomes 23:30–24:30. Times are not shifted to today's calendar date.
11. **Model training.** Run `python run_model_training.py`. Pass: it trains amendment, exposure and issue models; writes all three models, both feature schemas and `output/model_metrics.json`; the metrics include training/held-out rows, scores, minimums and `meets_minimum`; it says whether each reaches its minimum and finishes normally.
12. **Scores are meaningful.** Pass: amendment scores in the live queue cover LOW, MEDIUM and HIGH, the scores are not all the same (standard deviation is above 0.05), bands use rounded scores (LOW below 0.30, MEDIUM from 0.30, HIGH from 0.60), and the amendment score is not described as a code breach. Exposure predicts under-18 viewers in thousands; `audience_all` is not used as an input.
13. **Knowledge index rebuild.** Run `python run_knowledge_indexing.py`. Pass: it reads 16 documents, finds 150 sections, stores passages and reports the store location. Passage count may vary with chunk size.
14. **Rules come from their own codes.** Pass: the indexer prints each of the five category rules under the right code name. Compare the values with the answer-key table above. Betting alone has the live-sport exception.
15. **Service and dashboard start.** Start the API and dashboard as above. Pass: service announces port 8081 and `/docs`; dashboard announces port 8080; browser shows a sign-in page. If the API reports a missing model, schema, live file, index or database, it must name the missing item and the setup step that creates it.

B. Sign-in, API, queue and role permissions

16. **Correct sign-in.** Use a valid demo account. Pass: response contains `username`, `display_name`, `role`, `assigned_agencies`, `session_token`, `pages` and `areas`; values match that account's role.
17. **Wrong sign-in.** Try a deliberately wrong password, then an unknown username. Pass: both return HTTP `401` and the same sign-in-failed message; neither reveals which username exists.
18. **No token.** Make `GET /queue?product_category=Alcohol` without the header. Pass: HTTP `401`, sign-in required; the service refuses it even if the dashboard page is hidden.
19. **Unknown/stale token.** Change one character of the session token, or sign out then reuse it. Pass: HTTP `401`; the protected request does not return campaign data.
20. **Role pages for Controller.** Sign in as Priya or Owen. Pass: the navigation matches the `pages` list returned at sign-in and includes Clearance chat, Clearance queue, Campaign detail, Decision queue and Audit trail.
21. **Role pages for Executive.** Sign in as Tom. Pass: the navigation matches the returned `pages` list and includes Clearance chat, Clearance queue and Campaign detail; Decision queue and Audit trail are refused by the service (`403`).
22. **Role pages for Liaison.** Sign in as Lucy or Nadia. Pass: navigation matches the returned `pages` list; the brief's work page is Clearance chat. Attempts to open queue, campaign detail, decision queue or audit data return `403` from the service. If a Home page is offered, it must describe the signed-in role without granting extra API access.

23. **Full queue.** As a desk role, use the all-category option shown in `/docs`. Pass: 240 unique campaigns appear once, positions begin at 1 and queue order is exposure high-to-low, then first transmission, then campaign reference alphabetically.
24. **Category filters.** Try Alcohol, Betting and gaming, Less healthy food, Consumer credit and Medicines and health. Pass: every returned row matches the filter, and its queue position matches its position in the full queue.
25. **Bad filter and bad reference.** Use a category not listed by `/docs`; then GET `/queue/CLR-NOT-REAL-0000000`. Pass: invalid category returns `422`; missing campaign returns `404` with the reference in the error.
26. **Campaign detail response.** GET `/queue/CLR-QSC-0042117`. Pass: detail includes campaign facts, five code rules, notes and issue, forecast schedule, every spot result, summary and exposure. No language model call is needed to fetch detail.

C. Broadcast schedule and category rule checks

27. **Worked-example first check.** In the detail response for `CLR-QSC-0042117`, confirm 10 booked spots: 4 compliant and 6 breaching. Expected breach counts: WINDOW 3, INDEX 1, CHILDREN 0, FREQUENCY 1, SEPARATION 1.
28. **Worked-example order and decisions.** Pass: first transmission is Saturday 18:20; the decisions open are `CLEAR_WITH_RESTRICTIONS` and `REJECT`. `CLEAR` is not open because some spots breach.
29. **Worked-example restriction outcome.** For this example, if `CLEAR_WITH_RESTRICTIONS` is chosen: spot references ending 03, 04, 07 and 09 are cleared; the other six are withdrawn. The notice names each withdrawn spot once; the certificate names each cleared spot once and none of the withdrawn spots.
30. **Centre break placement.** Pick a spot whose break time is strictly between one programme's start and end. Pass: it is linked to that one programme and identified as an in/centre break using the terminology shown by `/docs`.
31. **End break placement.** Pick a spot whose time is exactly when one programme ends and the next starts. Pass: it is linked to both programmes, in before/after order, and checked against both indices and both child marks.
32. **Impossible schedule location.** Use the data shown in the live schedule to identify a break at a broadcast-day edge or in a schedule gap, if one is available in the SME test set. Pass: it is refused; it is not moved to a nearby programme. The supplied weekly schedule should otherwise have no gaps.
33. **Audience threshold boundary.** Check spots adjacent to programmes with indices below and equal to a category's threshold. Pass: below is not an index breach; equal or above is a breach. An end break breaches if either adjacent programme meets the threshold.
34. **Children's rule difference.** Compare Medicines and health with Alcohol, Betting, food or credit. Pass: a medicines spot in the centre of a marked children's programme breaches CHILDREN; an end break merely adjacent to it does not (but still receives the index test). For the other four codes, both in and adjacent are covered.
35. **Betting exception only.** A betting spot in restricted hours can be exempt only in a centre break of its own live-sport programme, more than 10 minutes after the start and more than 10 minutes before the end. Pass: an end break, studio sports show or boundary at exactly 10 minutes does not get the exception. Other categories never get it.
36. **Restricted hours.** Compare a spot inside the windows with one at the end boundary. Pass: Betting and food are restricted 05:30–21:00; credit 06:00–19:00, including live sport; alcohol and medicines have no window. The scheduling rules say a break at the start is inside and a break at the end is outside.
37. **Daily limit.** Count one campaign's spots per channel and broadcast day, in transmission order. Pass: only spots beyond that code's limit breach FREQUENCY; every booked spot counts even if it already breaks another rule. A different channel or broadcast day has a separate count.
38. **Minimum spacing.** Compare successive spots on the same channel and broadcast day. Pass: exactly the required minutes is allowed, less than that breaches SEPARATION, spots in one break are zero minutes apart, and a spot is measured from the previous booked spot even when that previous spot breaches.

39. **Breaches listed in order.** For a spot breaking multiple rules, pass: all matching codes appear once in this order: WINDOW, INDEX, CHILDREN, FREQUENCY, SEPARATION. A spot breaking none has an empty breach list.
40. **Exposure formula.** Pass: exposure adds forecast under-18 viewers for breaching spots only; at an end break it uses the larger of its two programme forecasts; non-breaching spots do not add exposure; the total is rounded once to one decimal place; no breaches means zero.
41. **Queue tie order.** Find equal exposure entries if any exist. Pass: earlier first transmission comes first, then alphabetically smaller campaign reference. If no tie exists in the loaded queue, record “not present in this data set” rather than inventing a result.

D. Human decision gate, records and documents

For these tests, use one fresh campaign per decision. An Executive opens the assessment; a Controller who did not open it decides. Capture `assessment_id` from the response and use that ID in the next request.

```
$AssessmentBody = @{
  campaign_reference = "CLR-QSC-0042117"
  executive_request = "Please review the copy and every booked spot against the campaign category code."
} | ConvertTo-Json
$Assessment = Invoke-RestMethod -Method Post -Uri "$Api/assessments" `
  -Headers $Headers -ContentType "application/json" -Body $AssessmentBody
$Assessment | Format-List *
$AssessmentId = $Assessment.assessment_id
Invoke-RestMethod -Method Get -Uri "$Api/clearances/$AssessmentId/calls" -Headers $Headers |
  Format-List assessment_id, call_count, expected_call_count, total_latency_milliseconds
```

Use the sample only if it has not already been assessed. The known answer for that sample’s open decisions is listed in case 28.

42. **Open an assessment.** Pass: POST `/assessments` returns HTTP `201`; response status is `PENDING_DECISION`; it contains the recommendation and decisions open. The five assessment agents have each made one call (five calls before the gate).
43. **Pause before a human decision.** Before submitting a decision, GET `/clearances?status_value=PENDING_DECISION` and the call budget. Pass: campaign is waiting; no spot has been cleared or withdrawn; no agency notice or certificate has been sent; call count is 5.
44. **Assessment only once.** POST `/assessments` again for the same campaign. Pass: HTTP `409`, already-assessed message; it does not create another run.
45. **Invalid assessment request.** Send a too-short `executive_request` or a campaign reference that is not in the 240-row queue. Pass: short request gives `422`; missing campaign gives `404`; neither begins a run.
46. **The opener cannot decide.** Open a fresh campaign as one Controller, then attempt to decide it as that same Controller. Pass: HTTP `403` with the own-assessment refusal. No decision is recorded. A different Controller can decide it.
47. **Only open decisions are offered.** On the worked sample, submit `CLEAR` once before the real decision. Pass: HTTP `409`, decision-not-open; the assessment is still pending. This sample allows restriction or rejection only.
48. **Reject path.** On a fresh campaign, have a different Controller submit `REJECT` with a meaningful reason. Pass: status `REJECTED`; zero spots cleared; every spot withdrawn; no restriction notice or clearance certificate; no more model calls after the five pre-gate calls (total 5).
49. **Clear path.** On a fresh campaign with zero breaching spots, submit `CLEAR`. Pass: status `CLEARED`; every spot cleared; no spots withdrawn; a certificate is written; no restriction notice; one certificate-writing call (total 6).
50. **Clear-with-restrictions path.** On a fresh mixed campaign, submit `CLEAR_WITH_RESTRICTIONS`. Pass: status `CLEARED_WITH_RESTRICTIONS`; compliant spots only are cleared and breaching spots withdrawn; notice is written before certificate; two document-writing calls (total 7).

51. **Documents match the decision.** For a restricted clearance, every withdrawn spot appears exactly once in the notice with channel/day/time and breach rules; every cleared spot appears exactly once in the certificate; no withdrawn spot appears as cleared. Both use the campaign category's own code. For a rejection, neither document exists.
52. **Call budget and call records.** GET `/clearances/{assessment_id}/calls`. Pass: before decision expected count 5; final totals are 5 for REJECT, 6 for CLEAR and 7 for CLEAR_WITH_RESTRICTIONS. Each recorded call has agent name, model name, prompt/response character counts and latency, in call order.
53. **Decision reason and status.** GET `/clearances?status_value=REJECTED` (or the status actually reached). Pass: only matching assessments appear; decision, reason, Controller display name/role, time, cleared/withdrawn counts and status are saved.
54. **Duplicate decision.** Submit a second decision after one is recorded. Pass: HTTP `409` already-decided; saved outcome does not change.
55. **Decision persistence after restart.** On a fresh pending assessment, stop the API with Ctrl+C, restart `python run_api_service.py`, and fetch the pending assessment again. Pass: it is still pending and the same assessment ID and run are present; a different Controller can finish it once.

Decision call totals are from the brief's agent flow: five calls before the gate; CLEAR writes one certificate, CLEAR WITH RESTRICTIONS writes a notice and certificate, REJECT writes neither.

E. Clearance chat and confidentiality

Ask each question below as a separate message. Each answerable policy reply must name its question type, have `campaign_reference: null`, not be refused, and cite the appropriate guide/code in `source_documents`. For code-rule questions the service may offer all five code passages; the answer must still use the correct category's row only.

56. **Alcohol code retrieval.** Ask: "What hours apply to alcohol advertising, what is its under-18 index threshold, and how do its children, daily limit and spacing rules work?" Pass: any time; index 120; children in or adjacent; six per channel/day; 20 minutes; source includes `Code-Alcohol.pdf`.
57. **Betting code retrieval.** Ask: "What is the betting and gaming restricted window, the under-18 threshold, and the live-sport exception?" Pass: 05:30–21:00; index 110; only a centre break more than 10 minutes inside both ends of its own live-sport coverage is exempt; source includes `Code-Betting-And-Gaming.pdf`.
58. **Less healthy food retrieval.** Ask: "What hours, index threshold, children's rule, daily limit and spacing apply to less healthy food?" Pass: 05:30–21:00; index 120; in/adjacent; eight per channel/day; 10 minutes; no live-sport exception; source includes `Code-Less-Healthy-Food.pdf`.
59. **Consumer credit retrieval.** Ask: "What is the consumer credit window and the audience, children, daily-limit and spacing rules?" Pass: 06:00–19:00 including live sport; index 125; in/adjacent; five per channel/day; 15 minutes; source includes `Code-Consumer-Credit.pdf`.
60. **Medicines retrieval.** Ask: "Can medicines air at any time, what is the audience threshold, and does the children's rule cover an adjacent end break?" Pass: any time; index 150; the children's rule covers a centre break in a marked children's programme, not merely an adjacent end break; 10 per channel/day; five minutes; source includes `Code-Medicines-And-Health.pdf`.
61. **Scheduling document retrieval.** Ask: "How does Sunday 05:40 move onto the broadcast clock, and what does an end break mean?" Pass: Saturday 29:40; end break is exactly between two programmes and is adjacent to both; source includes `Scheduling-And-Audience-Rules.pdf`.
62. **Advertising policy retrieval.** Ask: "What decisions can a Clearance Controller make, and when is CLEAR WITH RESTRICTIONS allowed?" Pass: CLEAR only when no spot breaches; CLEAR WITH RESTRICTIONS only when some spots comply and some breach; REJECT is always available; a machine recommendation does not clear or withdraw a spot. Source includes `Advertising-Clearance-Policy.pdf`.
63. **Copy guide retrieval: gambling.** Ask: "What should an executive check when a betting advert urges people to bet now?" Pass: identifies urgency/pressure, misleading win claims and safer-gambling message through the whole spot; source includes `Guide-Gambling-Inducements.pdf`.

64. **Copy guide retrieval: financial promotions.** Ask: “What must be shown with a cost-of-credit figure?” Pass: representative APR and representative example, at least as prominently as the cost figure; source includes `Guide-Financial-Promotions.pdf`.
65. **Copy guide retrieval: health claims.** Ask: “What evidence should be checked for a medicine’s health claim?” Pass: product licence/indication and supporting evidence for claims, onset and duration; source includes `Guide-Health-Claims.pdf`.
66. **Copy guide retrieval: comparative claims.** Ask: “What should be checked before saying a product is better than a competitor?” Pass: like-for-like basis, current evidence, identifiable comparison and fair treatment; source includes `Guide-Comparative-Claims.pdf`.
67. **Copy guide retrieval: environmental claims.** Ask: “What evidence is needed for an ‘eco-friendly’ or carbon-neutral claim?” Pass: claim scope is clear and evidence/independent verification covers the claim; source includes `Guide-Environmental-Claims.pdf`.
68. **Copy guide retrieval: price and offers.** Ask: “What should an executive check for a sale price or promotion?” Pass: genuine price/promotion, significant terms beside the headline and readable for long enough; source includes `Guide-Price-And-Offer-Claims.pdf`.
69. **Copy guide retrieval: alcohol/social success.** Ask: “Can an alcohol advert show a drink making somebody more popular?” Pass: no; the drink must not be linked to popularity, confidence or social success; source includes `Guide-Alcohol-And-Social-Success.pdf`.
70. **Copy guide retrieval: appeal to under-18s.** Ask: “What should an executive check when an advert uses cartoon characters or a celebrity?” Pass: whether the whole treatment appeals particularly to under-18s and whether a celebrity has a strong young following; source includes `Guide-Appeal-To-Under-18s.pdf`.
71. **Follow-up remembers context.** First ask the betting-threshold question (case 57), then ask “Does that exception work at the break between two programmes?” Pass: understands “that” means betting; answer says no, an end break is never within live sport coverage; the two turns remain in the same signed-in user’s conversation.
72. **Out-of-scope question refuses.** Ask: “What is the capital of France?” Pass: `was_refused` is true, answer is the configured refusal, sources are empty, and no outside fact is guessed.
73. **Casebook stays excluded.** Ask for an outcome from a past case in the casebook. Pass: assistant does not quote or rely on `Past-Clearances-Casebook.pdf`; if the remaining policy documents do not answer it, it refuses.
74. **One chat call per question.** Ask one new question and check it appears once in your own chat turns and once in the audit chat log. The product requirement is one language-model call for that question; the chat response itself does not report a model-call count, so verify the one generation in the approved model-provider request/usage log when the SME requires direct evidence.
75. **Liaison sees own agency status.** As Lucy, ask about assessed campaign `CLR-QSC-0042117` after its decision. Pass: status/time and decision, cleared/withdrawn counts and documents sent are given; no score, notes, breach details, audience index or staff name is exposed. For an unassessed QSC campaign, it says not assessed.
76. **Liaison ownership boundary.** As Lucy, ask about a real campaign from an agency outside QSC/HWK/PVT/FWL, then ask about `CLR-NOT-REAL-0000000`. Pass: both give the same not-found message, so the answer does not reveal that the other campaign exists. Repeat with Nadia and an agency outside her assignments.
77. **Chat history belongs to the user.** GET `/chat/turns?turn_limit=10`. Pass: only the signed-in user’s latest turns are returned, newest first, no more than ten; the chat page displays them oldest first. Another user’s questions do not appear.

F. Dashboard and audit trail

78. **Sign-in and sign-out screens.** Before signing in, only the sign-in form is shown. After signing in, dashboard shows display name and role (and Liaison agencies). Sign out; pass: session ends and the protected API token

no longer works.

79. **Queue screen.** Pass: top counts use the whole queue; filter changes results through the service; table shows position, reference, advertiser, category, spot counts, first transmission, exposure and amendment score/band. Selecting a campaign opens that campaign's detail.
80. **Campaign detail screen.** Pass: header shows category, score/band, decisions open, status and queue position; broadcast-week chart uses the broadcast clock; all five rules are shown beside their own code; every spot is in transmission order with programmes and results; copy and note issue are visible.
81. **Assessment screen and one-time rule.** Pass: a sufficiently long request starts one assessment and shows its recommendation awaiting a Controller; a second attempt on the same campaign shows the already-assessed message rather than starting another run.
82. **Decision queue screen.** Pass: only pending assessments appear; the selected item shows recommendation, category/issue, counts, exposure, assessment, breaching spots/rules and who opened it. The opener sees the refusal in place of the decision form. Other Controllers see only decisions open and must enter a reason.
83. **Audit trail screen.** Pass: selected record shows status, recommendation, opener, decision and reason, cleared/withdrawn spots, documents and each model call against expected count/total latency. The chat tab shows recent questions with asker, role, type, campaign reference, refusal and sources.
84. **No hidden-page security.** Try the protected endpoint directly as a role without access, even if that button is absent. Pass: service returns `403`; hiding a dashboard button alone is not enough.

Quick PowerShell decision request

Use only after the assessment is pending and signed in as a different Controller from the person who opened it. Replace the example ID with the `assessment_id` you captured. Use only a decision listed in `decisions_open`.

```
$DecisionBody = @{
  decision = "REJECT"
  decision_reason = "I reviewed the assessment and confirm the decision for this campaign."
} | ConvertTo-Json
Invoke-RestMethod -Method Post `
  -Uri "$Api/clearances/REPLACE-WITH-ASSESSMENT-ID/decision" `
  -Headers $Headers -ContentType "application/json" -Body $DecisionBody
```

Expected for the example campaign, if you choose `REJECT`: status `REJECTED`, 0 cleared and 10 withdrawn, no notice or certificate, total call count 5. For a different campaign, compare counts to its own detail response.

How to check expected errors

The easiest way is `/docs`: execute the request and read its HTTP status and `detail`. Expected statuses are:

Situation	Expected status
Wrong credentials, missing/stale token	401
Signed-in role cannot reach a page/endpoint; opener tries to decide own assessment	403
Campaign or assessment ID does not exist	404
Campaign already assessed, already decided, or decision is not open	409
Missing/too-short/invalid request field or invalid category/status	422
Successful ordinary request	200
Assessment created	201

For PowerShell, a deliberate error throws a message. Read the numeric HTTP status and returned `detail`; do not mark a failed request as a pass merely because an error appeared. The status and reason must match the table and scenario.

Manual result sheet

For every numbered case, record: **Case number | Pass/Fail/Not present | Actual result or value | Campaign reference (if any) | Tester/date | SME initials**. Never include passwords, API keys or session tokens. A case that needs a tie, schedule gap or other special input not present in the supplied 240 campaigns should be marked **Not present** and tested with an SME-provided fixture, not simulated by editing live data.

Source of expected answers

This guide is based on `BreakCheck_AI.pdf` (especially its problem statement, domain rules, expected outputs and implementation steps) and the policy/reference PDFs named by that brief under `Project/data/docs/`. The five code values in the table are transcribed from the category code PDFs; operational wording and decision rules come from `Advertising-Clearance-Policy.pdf` and `Scheduling-And-Audience-Rules.pdf`. This guide does not use application source code as an answer key.